11-768 HW2: Evaluating a Data-Visualization Agent

Independent, non-credit reproduction

Russell (xqzr2000)

November 2026

Deviations from the official setup (read first). This is a self-study reproduction, so it does not use the course's Modal deployment. The judge is **GPT-6.1 Terra** (`gpt-6.1-terra`, OpenAI chat completions, `reasoning_effort=none`, flex tier, temperature 0), not Qwen3-VL-30B-A3B. The three fixed agents are replaced by **DeepSeek Flash**, **GPT-5.6 Luna** and **GLM-5.3 Flash**, run by our own runner (`workflow/sandbox_generate.py`) that keeps the course contract: mini-swe-agent's prompt, a single `bash` tool, 20 steps, 120 s per command, 15 minutes per run, and submission by `echo COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT`. Each run executes in a fresh Docker container from our Assignment 1 sandbox (no network, all capabilities dropped, CPU/memory/PID limits). The image is pinned by digest and hash-locked; API keys never enter the container. All MCCs below are therefore for a much stronger judge than the graded one and are not comparable to course scores. Total judge spend for every number in this report was about $5 (`scripts/judge_cost.py`).

Reproduction: `validator.runner` (we added a `-workers` flag) and `workflow score` as in `ASSIGNMENT.md`. Every prediction file named below is in `output/`. Working notes: `docs/report-notes.md`.

1 Part 1: Examine Seed Agent Runs

1.1 Comparison of Ground-Truth and Validator Labels

The baseline (`validator/baseline.py`, unchanged) asks four separate YES/NO questions per run with the full trajectory, inputs and figure (402 calls, 3.3M prompt tokens, $1.71).

Family	MCC	TP	TN	FP	FN	support +/-
execution_failure	0.5945	8	93	5	4	12/98
wrong_data	0.2363	22	27	46	3	25/73
wrong_chart	0.6286	66	18	9	5	71/27
hard_to_read	0.6345	29	52	7	10	39/59
macro	**0.5235**					

Disagreements concentrate in `wrong_data`. 46 of its 73 negatives are flagged, so recall is high and precision is near chance. The remaining errors cluster on a handful of runs whose agents struggled (sunburst, Sankey and voxel tasks), where one wrong execution verdict removes the other three families. One seed fact matters for the design: the 12 runs labelled `execution_failure` are exactly the 12 runs with no `figure.png`.

1.2 Validator Error Type 1: execution judged from the story, not the artifact

False positives `026`, `047`, `081`, `191` and `205` all saved a decodable PNG, yet the judge answered that the agent "exhausted its limits" (`026`, `047`), "exited after repeated formatting errors" (`191`, `205`) or that the image was "not the requested 3D voxel plot" (`081`). In each case it read trouble in the trajectory (exit status `LimitsExceeded`/`RepeatedFormatError`, failed intermediate scripts) or a wrong chart as "no valid figure". Because the baseline returns early on `execution_failure`, these five runs also lose 3 `wrong_data`, 5 `wrong_chart` and 3 `hard_to_read` true positives. **False negatives** `001`, `145` and `171` have no figure but were judged acceptable. `103` also has none, but the judge reported wrong data, design and legibility for an image that does not exist. The baseline never checks the one fact that decides the label: whether the file exists and decodes.

1.3 Validator Error Type 2: per-family YES/NO questions conflate families

Asked "does the plotted data differ from what was requested in any way?", the judge answers YES whenever anything is wrong. Its own evidence often says the data is fine: `021` and `090` ("the plotted data, medians, and outliers match the request, but the chart is missing an x-axis label"); `013`, `106`, `130` (forbidden colormap); `009`, `059` (tick labels not rotated); `131`, `143` (wrong subplot grid); `080` (missing value labels). The leak also runs the other way. Swapped values (`077`, `204`, `216`) and filtered-out rows (`008`, `069`) are reported as `wrong_chart`. Several readability problems are seen but not filed as such: `175` (legend covers bar tops), `173` (annotation hides the curve), `129` (white labels on white) and `213` (colliding captions).

1.4 Proposed Validator Changes

1. **Artifact-gated execution check.** Decide `execution_failure` deterministically from `figure.png` (missing, not a PNG, or undecodable), never from the narrative, and always run the other judges when a valid figure exists.
2. **Requirement checklist with family tags, plus a separate readability pass.** One structured call lists every requirement in the instruction, tags it `data` or `design` using the official definitions, and marks it met / violated / unclear. A family fires only when a requirement of its kind is violated. Legibility is judged in a separate, image-first call against the official `hard_to_read` checklist, told that missing or wrong elements are not legibility problems.

2 Part 2: Improve Your Validator

2.1 Implemented Changes

Both changes are in `validator/solution.py`.

- **Deterministic:** `judge_execution` opens the PNG with Pillow (`verify` + full `load`). No model call is made for runs without a valid figure.
- **Measured image facts (deterministic, given to the judge):** pixel size, PNG DPI metadata, implied figure size in inches, and a blank-image flag. Requirements like "1000×1000 pixels" or "120 dpi" are then checked against facts instead of guessed from a downscaled image.
- **Model, checklist call:** the instruction, input files (each clipped to 20k characters), a compact transcript (agent text and commands in full, tool output clipped to 1.5k characters), the image facts and the figure. The reply is a strict JSON schema (`requirement, kind`∈{`data`, `design`}, `status, evidence`). `wrong_data`/`wrong_chart` are derived deterministically from violated items, and their evidence lists the violated requirements.

- **Model, readability call:** the instruction (for context) and the figure only, with no code or transcript, so legibility is judged from pixels. The reply is a list of issues in six categories mirroring the official list, each rated `minor`/`major`. Only `major` counts.

The validator makes two judge calls per run with a figure (about $1 per seed pass at flex rates), compared with four for the baseline.

2.2 Seed-Set Performance vs. Baseline

`output/pre_iteration_seed_predictions.json`. We ran it twice; pass 1 gave macro 0.8657 (`wrong_data` 0.7045), so judge noise is about ±0.01 macro even at temperature 0.

Family	Baseline	Part 2	TP	TN	FP	FN
execution_failure	0.5945	**1.0000**	12	98	0	0
wrong_data	0.2363	**0.7406**	23	64	9	2
wrong_chart	0.6286	**0.9284**	68	27	0	3
hard_to_read	0.6345	**0.8297**	33	57	2	6
macro	0.5235	**0.8747**				

2.3 Qualitative Analysis of Targeted Error Types

Error type 1 is eliminated on the seeds. All 12 figure-less runs are caught with a concrete message and no model call, and `026`, `047`, `081`, `191`, `205` are now judged on their figures: e.g. `047` gets `wrong_data` and `wrong_chart`, as labelled.

Error type 2 is largely fixed. `wrong_data` false positives fall from 46 to 9. `021` and `090` now receive only `wrong_chart` ("missing x-axis label" is tagged `design`), and `013`, `106`, `130` lose their spurious `wrong_data`. `wrong_chart` false positives fall from 9 to 0: the swapped values in `077`, `204`, `216` are now `wrong_data` only, as labelled.

A remaining false positive (wrong_data, `068`). The task asks for a dotted $y = x$ line on a Mercator-style plot; the human label is `wrong_chart` only (solid line, x limits ±200). Our judge also reported `wrong_data`: "the saved script plots `s=t*pi/360` against `t`, so at longitude 180 the plotted latitude is about 1.57". It reasoned from the code's unit conversion rather than from what the axes display. Giving the judge code catches real data bugs, but it also lets it "find" errors the rendered figure does not show. Most of the 9 remaining `wrong_data` FPs have this shape (`086`, `115`, `161`, `163`, `203`, `206` all cite script internals). What is missing is a rule that a data violation must be visible in, or provable from, the figure.

A remaining false negative (hard_to_read, `175`). The human label says "the legend covers the tops of bars, so their heights cannot be read". The trajectory places the legend with a bare `ax.legend()`. Our judge reported the 3D/2D design violations but no legibility issue. The readability call asks for a list of problems but never makes the judge look at what each overlay sits on, and at full-figure resolution a legend over the tops of thin bars is easy to miss. `086` (pale edgeless bars) and `129` (white labels on white) are the same pattern: the problem is small relative to the image. Part 4 targets exactly this.

3 Part 3: Design New Evaluation Tasks

3.1 Anticipated Generalization Weaknesses

1. **Large-input aggregation** (`task_class = large-input-aggregation`). The validator clips each input file at 20k characters, and the judge cannot execute code. When the plotted values are aggregates (sums, shares, medians) over thousands of rows, the judge can neither see all rows nor do the arithmetic. It must either trust the agent's code or mark the data requirement `unclear`, which our derivation treats as not violated. We therefore expected missed `wrong_data` errors.
2. **Fine visual detail** (`task_class = fine-visual-detail`). Exact hex colours, dash patterns, marker shapes, log-axis formatting, and long rotated tick labels near the image edge are near the resolution limit of a vision model that sees a downscaled figure. We expected both missed `wrong_chart` details and missed `hard_to_read` clipping and overlap.

3.2 Task Design

All inputs are synthetic and seeded (`scripts/make_part3_inputs.py`), so nothing published here leaks a real dataset.

- `quarterly-revenue-by-region` (aggregation). `transactions.csv`, 3,000 orders (119 KB, six times the clip). Revenue per region and calendar quarter, units × unit_price/1000. A grouped bar chart with fixed region order, exact title and labels, y from 0, 10×6 in at 100 dpi. A wrong aggregation (e.g. summing units) produces a plausible chart whose error is invisible without recomputation.
- `top-products-unit-share` (aggregation). Same CSV. Each product's share of *all* units, top 8 only, horizontal bars sorted largest-on-top, one-decimal % labels. Tests normalization against the full denominator, filtering and ordering.
- `sensor-daily-median` (aggregation). `sensors.csv`, 12,096 five-minute readings from 3 sensors with 4% blank dropouts (311 KB). Daily medians for S1 and S2 only (exclude S3, ignore blanks), with a min–max band for S2. Tests filtering, missing-value handling and per-day grouping.
- `styled-decay-curves` (visual detail). Three analytic curves with exact hex colours, `-`/`--`/`-.` styles, `o`/`s`/`^` markers every 20th point, line width 2, log y in $[10^{-5}, 2]$, dotted grid, legend lower left, 8×5 in at 150 dpi.
- `dense-department-budgets` (visual detail). 30 bars labelled with long department names (up to 51 characters), rotated 60°, right-aligned, 8 pt, all fully visible and non-overlapping on a 12×6 in canvas. Clipping and overlap at the image edge are the expected failure modes.

3.3 Agent Runs and Labeling

Each task was run once with each agent (15 runs, `runs/`). All 15 submitted a figure within 3–7 model calls, about 6 minutes in total, for under $1. To label, we replayed every run's bash commands in a scratch directory under the course's savefig hook (`harbor/example/tests/figure_manifest.py`). Every replay was pixel-identical to the delivered figure, so the resulting JSON manifest is exactly what was drawn. We compared plotted values, order, tick labels, colours, line styles, markers, scales and limits against answer keys computed independently with Python's `csv` module, and then inspected every figure by eye. Claude drafted the labels with this evidence and Russell reviewed them (`labels.json`).

Outcome: all 15 runs plot the correct data with the requested design. The only errors are `hard_to_read`:

- `dense-department-budgets__glm`: the rotated names run off the bottom-left edge ("...t of Transportation and Infrastructure") because the agent used a fixed `subplots_adjust(bottom=0.32)`.
- `dense-department-budgets__luna`: the agent wrapped each name onto 2–4 lines "to keep the complete names readable" and then rotated them; the wrapped blocks overprint their neighbours.
- `sensor-daily-median__{deepseek,glm,luna}`: the default upper-left legend hides the upper edge of the S2 band near 06-02, which is that day's maximum, about 32.1 °C.

Ambiguous decisions. (i) The sensor legend hides only a small part of the band. We labelled it `hard_to_read` because the seed labels count a legend hiding part of a curve (`074`). (ii) Luna's wrapped names are still "the full department names", so we did not add `wrong_chart`; the overlap is legibility. (iii) For GLM's clipped labels we chose `hard_to_read` (partly cut) rather than `wrong_chart` (entirely outside), per the official rule.

3.4 Validator Performance on New Tasks

`output/pre_iteration_authored_predictions.json`:

Family	MCC	TP	TN	FP	FN	support +/-
execution_failure	0 (n/a)	0	15	0	0	0/15
wrong_data	0 (n/a)	0	15	0	0	0/15
wrong_chart	0 (n/a)	0	14	1	0	0/15
hard_to_read	0.5547	2	10	0	3	5/15
macro	0.1387					

Three families have no positive labels, so the scorer reports 0 by convention. **Agent vs. validator weaknesses.** The aggregation class did not expose validator weakness 1, because no agent produced a wrong aggregate: these agents are much stronger than the course's 3B–30B models. The validator's "acceptable" on those 9 runs is correct, but it was not *verified*. It saw only the first and last 10k characters of each CSV, so a wrong sum would very likely also have been called acceptable. We show in Section 3.5 that Harbor's deterministic verifier *does* catch exactly such an error. The visual-detail class exposed agent weaknesses (GLM's fixed margins, Luna's wrapping) and two validator weaknesses. First, all three sensor occlusions were missed, a small overlay at full-figure resolution. Second, one false positive: on `dense...__luna` the judge filed "ensure no x-axis tick labels overlap" as a violated *design* requirement. My instruction stated a legibility property as a requirement, and the checklist had no way to say "this is legibility". Both the exact styles and the large-scale values were judged correctly in all 15 runs.

3.5 Harbor Packaging

`scripts/build_harbor_tasks.py` fills each `harbor/tasks/<id>/` from the `workflow package` scaffold. It writes an exact sidecar schema for `plotted_values.json` in `instruction.md` (the seeded prompt is untouched), a reference `solve.sh`, and a verifier. The verifier (`harbor/shared/verifier_core.py`, adapted from the course example) implements S1 (valid non-blank PNG, inputs unchanged by sha256, script left behind), S2 (re-run `plot.py` in a scrubbed copy under the savefig hook and assert on the artist-tree manifest: values, order, orientation, tick labels, title/labels, limits, scale, colours, line styles, markers, legend), S3 (the delivered PNG equals a fresh render) and S5 (the sidecar matches literal answer keys computed independently of the reference solution).

Rewards (`workflow check-submission`, real Harbor in Docker): [pending: filled from check-submission]

Mutant scoring 0: `quarterly-revenue-by-region/mutants/sum_units` sums *units* instead of revenue (×1000 so the bars stay on a similar scale). The chart looks right but East Q1 is 2694 instead of 74.732. It is rejected by `test_s5_sidecar_matches_key` and `test_s2_sixteen_vertical_bars_with_the_right_heights`.

Mutant scoring 1.0: `dense-department-budgets/mutants/unrotated_labels` has correct values, names, order, colour and size, but leaves the 30 names horizontal and drops `tight_layout`, so they overprint into one unreadable band and the outer ones run off the canvas. The manifest records tick *text*, not tick geometry or rotation, and nothing compares rendered text bounding boxes, so every assertion passes. This is a genuine `hard_to_read` error that only a model looking at pixels catches.

Can a deterministic verifier decide each family for these tasks?

- `execution_failure`: **yes**. File existence, PNG validity, a non-blank canvas and a re-runnable script are all mechanical.
- `wrong_data`: **yes, for these tasks**, because each has a closed-form answer key and the manifest exposes the plotted numbers exactly. It is stronger than our VLM here, especially for large inputs.
- `wrong_chart`: **mostly**. Type, orientation, labels, limits, scale, colours, dash style and marker are in the manifest. Rotation, `markevery`, legend location and dotted-grid style were not checked, and some design properties (e.g. "3D", "hollow centre") need chart-type-specific readers.
- `hard_to_read`: **no, in general**. Overlap, clipping, occlusion and contrast are properties of the rendered layout. One could compute text bounding boxes after drawing, but "readable" is a perceptual threshold that any such rule only approximates. The second mutant shows the gap.

Why per-task verifiers cannot grade the private runs. A Harbor verifier encodes one task's answer key and its exact output contract (`plot.py`, `plotted_values.json`, fixed filenames and schema). The private runs come from tasks we never authored, with no answer keys, no sidecars, no re-runnable script contract, and no figure manifest; the validator receives only a trajectory and a PNG. Writing a verifier needs the task in advance, and grading needs a judgment that generalizes across unseen tasks. That is the job the VLM validator exists to do.

4 Part 4: Iterate on Your Design

4.1 Additional Improvement Motivated by Part 3

1. **A `legibility` requirement kind** that never counts as `wrong_chart`. Motivated by the `dense..._luna` false positive: instructions on private tasks may also phrase legibility as a requirement ("all labels must be visible"), and the official taxonomy puts the failure in `hard_to_read`.
2. **Overlay inventory and enlarged quadrants.** The readability call must first list every legend, colour bar, annotation, text box and value label with what lies beneath it, and it receives the four quadrants of the figure enlarged to about 1600 px in addition to the whole image. Motivated by the three missed sensor occlusions, and by the seed misses with the same shape (`175`, `213`, `129`, `086`).

4.2 Final Evaluation on Seed and New Tasks

First version (4a, `output/part4a_*`) counted any overlay that covers data or text as `hard_to_read`. On the authored runs it fixed all three misses (`hard_to_read` 0.5547→1.0, macro 0.1387→0.25, and the `wrong_chart` FP disappeared). On the seeds it over-fired: `hard_to_read` FN 6→2 but FP 2→12 (MCC 0.8297→0.7296; macro 0.8327, with `wrong_data` at 0.6727 within run-to-run noise).

Calibration (4b). The judge must also state whether an overlay *hides information a reader needs*. Labels drawn on their own segment and arrows that touch their target are declared intended placements.

Set	Family	Part 2	4a	4b	**Final**
Seeds (110)	execution_failure	1.0000	1.0000	1.0000	–
	wrong_data	0.7406	0.6727	0.6727	–
	wrong_chart	0.9284	0.9284	0.9012	–
	hard_to_read	0.8297	0.7296	0.7703	–
	macro	**0.8747**	0.8327	0.8360	–
Authored (15)	hard_to_read	0.5547	1.0000	1.0000	–
	wrong_chart FP	1	0	0	–
	macro	0.1387	0.2500	0.2500	–

On the authored runs `execution_failure`, `wrong_data` and `wrong_chart` have no positive labels (MCC 0 by convention), so only `hard_to_read` and the false-positive count carry information. Seed confusion counts for `hard_to_read`: Part 2 FP 2 / FN 6 (`086`, `129`, `161`, `175`, `209`, `213`); 4a FP 12 / FN 2; 4b FP 7 (`047`, `050`, `097`, `126`, `136`, `143`, `202`) / FN 4 (`046`, `086`, `129`, `161`). 4b fixed `175`, `209` and `213`, but it still fired on intended overlays (`097`'s deliberately placed max box) and on new borderline cases. The `wrong_data` difference between Part 2 (0.7406) and 4a/4b (0.6727, FP 9→11) is the same order as the run-to-run noise we measured (0.7045 vs 0.7406 for identical code); change 4 does not touch the data path except by offering the `legibility` kind.

Choosing the final validator. 4b wins on the 15 authored runs and loses 0.039 macro on the 110 seeds. The seed set is seven times larger, covers many more chart types, and is drawn from the same population as the private runs; the authored gain rests on 3 runs of one task that share a single failure. We therefore ship **Part 2 + change 4 only** (the `legibility` kind) and keep change 5 behind `OVERLAY_INVENTORY = False` in `validator/solution.py`, with the readability prompt byte-identical to Part 2. The *Final* column is that configuration (`output/final_seed_predictions.json`, `output/authored_predictions.json`); 4a and 4b are kept as `output/part4a_*` and `output/part4b_*`.

4.3 Qualitative Analysis of Self-Authored Tasks

On the self-authored runs, change 5 did exactly what it was designed for. In `sensor-daily-median__{deepseek,glm,luna}` the overlay inventory reported the upper-left legend covering the S2 band's upper edge, which Part 2 had called acceptable in all three, and `hard_to_read` went from 2/5 to 5/5 with no new false positives. Change 4 removed the only authored false positive: on `dense-department-budgets__luna` the overlap requirement is now tagged `legibility`, and the run receives `hard_to_read` only, matching the label. The lesson from the seeds is that a rule written to fix one failure on 15 runs must be checked against the broader set before shipping; legend-over-data is real, but most overlays on real charts are intended.

False positive with trajectory text (4a, seed `097`). The agent placed its maximum label with `plt.annotate(f'Max: {z_max:.3f}', xy=(x_max, y_max), xytext=(x_max + 0.2,`

`y_max + 0.2), ..., bbox=dict(..., fc='white', alpha=0.8))`. The 4a judge reported "the box covers part of the yellow and green filled contour bands". That is true, but the box was put there on purpose and hides nothing a reader needs, and the human label is acceptable. The rule "any overlay on data" had no notion of intent. 4b added one, yet `097` still fired: asked whether the semi-transparent box hides information, the judge said yes, so an intent judgment delegated to the model is only as calibrated as the model. This run is one reason the final validator ships without change 5.

False negative (4a/4b, seed `161`). The human label includes `hard_to_read`: "on the shared log axis the sine collapses into flat-topped blocks". The trajectory shows why: the agent set `ax2 = ax1` and plotted `np.sin(2*np.pi*t)` on the log-scaled axis. Our judge correctly reports the missing twin axis (`wrong_chart`) but not the legibility consequence. Our hypothesis: once a cause is filed as a design violation, the readability pass, which sees only pixels, interprets the blocks as an intended rendering of a clipped series rather than as unreadable data. Readability errors that are *consequences* of a design error are a blind spot of separating the two calls.

4.4 Further Changes

- **Infrastructure for this reproduction**: `validator/model.py` adapts to OpenAI's `max_completion_tokens`/`temperature` constraints, supports the flex tier, and logs token usage. `validator.runner -workers N` judges runs concurrently (default unchanged). `scripts/disagreements.py` lists per-family FP/FN with both evidences.
- **Not done, and what we would try next**: (i) a "visible-in-figure" rule for `wrong_data`, to cut the code-reasoning false positives (`068`, `086`, `161`); (ii) giving the judge deterministic summaries of large inputs (row counts, per-column totals) so aggregate checks are not blind; (iii) re-executing the agent's final script under the savefig hook inside a sandbox, as our labelling did, to give the judge exact plotted values. This is the single most powerful signal we found, but it executes untrusted code at grading time and therefore needs a sandbox the grader may not have.
- **Caveat on generalization**: every number here uses GPT-6.1 Terra. The validator's structure (deterministic execution gate, strict JSON checklists, derived families, image facts, crops) is model-agnostic, but the calibration of 4b (`hides_information`) relies on judgment that a 30B model may make less reliably.